

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación



[CC3067] Redes

Laboratorio 3

Protocolos de Enrutamiento

Nombre	Carné
Daniel Chet	231177
Cristian Túnchez	231359

Descripción de la práctica

El objetivo de este laboratorio fue implementar un protocolo de enrutamiento de estado de enlace (Link State) funcional entre múltiples nodos router, capaces de descubrirse entre sí, construir su tabla de enrutamiento mediante el algoritmo de Dijkstra, y reenviar datagramas de usuario protegidos con detección y corrección de errores Hamming(7,4) a través de la ruta más corta calculada.

El sistema se probó en dos fases: primero de forma local (seis routers en 127.0.0.1, topología fija A-F) para validar el protocolo de forma determinista, y después sobre una red privada virtual (Tailscale) interconectando los nodos de esta pareja con los de la

pareja compañera (Javier Linares, Dilary Cruz), confirmando interoperabilidad real entre dos implementaciones independientes del mismo contrato de protocolo.

Sobre el plano de datos ya funcional se retomó también el proceso de Cajero (ATM) y Servidor Bancario del Laboratorio 2 (login, retiro, logout), como aplicación de demostración que atraviesa la red entera —origen, routers intermedios y destino— usando la ruta óptima calculada por el plano de control.

1. Parámetros generales

Transporte	TCP (orientado a conexión, confiable)
Puerto	5000-5007 para routers (1 puerto TCP por nodo); 6000+ para hosts adjuntos (cliente, servidor, cajero, banco)
Formato	JSON serializado, codificado en UTF-8
Delimitador	\n al final de cada mensaje
Campo clave	"type" obligatorio en todo mensaje — identifica el tipo
Identificador	Cada nodo se identifica como ip:puerto (ver sección 3)

Por qué ip:puerto y no solo IP

En pruebas locales sobre 127.0.0.1 todos los nodos comparten la misma IP y solo se distinguen por el puerto. Por eso el identificador canónico de cualquier nodo (router o host adjunto) es ip:puerto, tanto en pruebas locales como en Tailscale, donde la IP por sí sola ya era única y el puerto no aporta ambigüedad adicional.

2. Tipos de mensaje

A. HELLO (Control)

Enviado al inicio y periódicamente a todos los vecinos configurados. Confirma que el vecino está vivo. No requiere respuesta explícita; el receptor registra al emisor como vecino activo. Cadencia: cada 10 segundos.

```
{
  "type": "HELLO",
  "from": "100.x.x.x"
}
```

B. LSA — Link State Advertisement (Control)

Construido una vez que el nodo ha confirmado sus vecinos activos. Se inunda a toda la red mediante flooding. El campo "origin" identifica al nodo que generó el LSA y no cambia en ningún salto. El campo "from" se actualiza en cada salto al IP del nodo que reenvía.

```
{
  "type": "LSA",
  "origin": "100.x.x.x",    <- nodo que generó el LSA (NO cambia)
  "seq": 3,                <- número de secuencia
  "links": {
    "100.y.y.y": 2,        <- vecino: costo del enlace
    "100.z.z.z": 5
  },
  "from": "100.x.x.x"      <- nodo que reenvía (actualizar en cada salto)
}
```

REGLAS DE FLOODING

- Mantener tabla de pares (origin, seq) ya procesados.
- Si (origin, seq) es nuevo → actualizar grafo local, reenviar a todos los vecinos excepto al que lo envió (campo "from").
- Antes de reenviar, actualizar el campo "from" a la IP propia.
- Si ya fue visto → descartar silenciosamente.
- Incrementar seq solo al generar un LSA propio, nunca al reenviarlo.

C. DATA (Datos)

Mensaje de datos entre el nodo origen y el nodo destino final. El frame {from, to, msg} se serializa a JSON y se protege con Hamming(7,4) aplicado por bloques de 4 bits sobre el frame completo, no solo sobre "msg". Los routers intermedios deben corregir errores sobre todos los bits recibidos antes de poder leer nada, pero solo leen el campo "to" para decidir el siguiente salto; nunca interpretan ni actúan sobre "msg". Únicamente el destino final usa el contenido de "msg". Los campos "from" y "to" no cambian en ningún salto.

```
{
  "type": "DATA",
  "len": 128,
  "bits": "0110011010..."
}
```

PIPELINE EN CADA ROUTER AL RECIBIR DATA

1. Recibir la cadena de bits (bits).
2. Corregir errores sobre todos los bits (Hamming, usando len).
3. Extraer los bits de datos ya corregidos.
4. Deserializar el JSON resultante y leer solo el campo to.
5. Consultar nodo_tabla_enrutamiento.csv con el valor de to.
6. Obtener IP y puerto del siguiente salto.
7. Re-serializar el mismo frame (sin tocar msg).
8. Aplicar Hamming(7,4) de nuevo sobre el frame completo (nueva bits).
9. Aplicar ruido a la trama ya protegida: cada bit se voltea independientemente según noise_probability.
10. Enviar por un socket TCP nuevo hacia el siguiente salto.

Aplicación Cajero/Banco (contenido de "msg")

El ATM y el servidor bancario del Laboratorio 2 se retomaron como dos hosts adjuntos ordinarios (rol "atm" y rol "bank"), cada uno conectado a su propio router-gateway, igual que cualquier otro host adjunto. No se abre una conexión TCP persistente entre ambos como en el Lab 2: cada comando/respuesta es un DATA independiente que viaja por el

plano de datos ya descrito, con el texto del comando en el campo "msg". Los routers intermedios nunca interpretan este texto, solo leen "to" como en cualquier DATA.

Comandos del cajero:

LOGIN|<tarjeta>|<pin>

WITHDRAW|<monto>

LOGOUT

Respuestas del banco:

LOGIN_OK|<mensaje>

LOGIN_DENIED|<mensaje>

WITHDRAW_OK|<monto>|<saldo>

WITHDRAW_ERROR|<mensaje>

LOGOUT_OK|<mensaje>

ERROR|<mensaje>

El banco mantiene la sesión autenticada indexada por la dirección (ip:puerto) del cajero remitente, ya que cada DATA es una entrega independiente sin estado de conexión persistente — a diferencia del Lab 2, donde la sesión vivía implícita en la conexión TCP.

3. Configuración de nodo

Cada nodo lee al inicio un archivo config.json con el siguiente formato:

```
{
  "name": "A",
  "ip": "100.x.x.x",
  "port": 5000,
  "noise_probability": 0,
  "neighbors": [
```

```
{ "ip": "100.y.y.y", "port": 5001, "cost": 2 },  
  { "ip": "100.z.z.z", "port": 5002, "cost": 5 }  
],  
"attached_host": { "role": "atm", "ip": "100.w.w.w", "port": 6000, "cost": 1 }  
}
```

4. Tabla de ruteo generada

Archivo `nodo_tabla_enrutamiento.csv` producido por el plano de control una vez convergido Dijkstra:

```
destination,next_hop_ip,next_hop_port,cost  
100.y.y.y:5001,100.y.y.y,5001,2  
100.z.z.z:5002,100.y.y.y,5001,7  
...
```

5. Tiempos y comportamiento

HELLO	Cada 10 s. Primer envío inmediato al iniciar el nodo.
LSA	Generar y floodear 5 s después del primer HELLO recibido.
Convergencia	Esperar 30 s antes de asumir que la tabla de ruteo es estable.
Hamming	Aplicar solo en plano de datos (mensajes DATA). HELLO y LSA van en texto plano.
Hilos	Routing y forwarding deben correr en paralelo (threads / async).
Expiración de vecino	Si no llega HELLO de un vecino activo en 3 ciclos de HELLO (30 s), se marca como caído y se regenera el LSA propio.

Recálculo de rutas	Tras la convergencia inicial, la tabla se recalcula de inmediato ante cualquier cambio del grafo (LSA nuevo, vecino caído/recuperado), con un respaldo periódico cada 15 s.
Debounce de LSA	Ante varios cambios de vecindario simultáneos, se espera 3 s antes de floodear un nuevo LSA propio, para no inundar la red.
Ruido	Se aplica después de Hamming, antes de cada envío DATA; 0 por defecto (noise_probability).

6. Puertos

Rango acordado entre las 2 parejas reales de este grupo de topología: 5000-5007 para routers, un puerto TCP dedicado por nodo. Como el equipo confirmó con el profesor que puede operar como 2 parejas reales y simular la tercera corriendo nodos adicionales, se usa el siguiente reparto, validado en una prueba de interoperabilidad por Tailscale:

Bloque	Asignado a
5000-5002	Pareja 1 — Daniel Chet / Cristian Tunchez (hasta 3 routers)
5003-5005	Pareja 2 — Javier Linares / Dilary Cruz (hasta 3 routers)
5006-5007	Pareja 3 simulada (nodos extra, ejecutados por quien corra la prueba)

6000+	Hosts adjuntos (cliente, servidor, cajero, banco), todas las parejas
--------------	--

7. Resultados

Convergencia y cálculo de rutas

Se levantó la topología local completa (routers A-F) con los tiempos de producción del protocolo (10 s HELLO, 5 s LSA inicial, 30 s espera de convergencia). Todos los nodos alcanzaron el estado Converged dentro del tiempo esperado y calcularon de forma independiente la misma ruta óptima entre los nodos con host adjunto (A→F):

Ruta calculada: A -> B -> D -> E -> F (costo total: 7)

coincidiendo exactamente entre el cálculo de Dijkstra del nodo origen y el del nodo destino, confirmando consistencia del grafo distribuido construido por flooding.

Prueba end-to-end del plano de datos (Cajero/Banco)

Con la topología convergida, se ejecutó el flujo completo del cajero (login, retiro de Q100.00, logout) contra el banco, atravesando los 5 nodos de la ruta óptima:

Paso	Resultado
LOGIN (tarjeta válida)	LOGIN_OK
WITHDRAW Q100.00	WITHDRAW_OK, saldo Q500.00 -> Q400.00
LOGOUT	LOGOUT_OK

Cada uno de los routers intermedios (A, B, D, E, F) registró explícitamente el reenvío del datagrama hacia el siguiente salto correcto: 6 eventos de reenvío por nodo (3 de ida, 3 de vuelta), confirmando que los 3 mensajes y sus 3 respuestas atravesaron realmente la ruta calculada, y no solo una entrega directa simulada.

Interoperabilidad entre parejas

Se estableció conexión sobre Tailscale entre los nodos de esta pareja y los de Javier Linares/Dilary Cruz, siguiendo el contrato común de protocolo. Los nodos de ambas parejas se reconocieron como vecinos válidos, intercambiaron HELLO y LSA, y lograron enrutar mensajes de datos entre routers de implementaciones independientes, confirmando que el formato de mensajes JSON (HELLO/LSA/DATA) es efectivamente interoperable entre dos códigos distintos.

Overhead y tolerancia a errores de Hamming(7,4)

Se ejecutó un experimento reproducible (semilla fija 3067) con 300 transmisiones por combinación de tamaño de frame (64-1024 bytes) y tasa de error por bit (0%, 0.1%, 0.5%, 1%, 2%), simulando volteo de bit independiente. Resultados representativos para un frame de 512 bytes (4096 bits de mensaje):

Tasa de error	Bits codificados	Overhead	Flips promedio	Recuperación exacta	Corregido
0%	7168	75%	0	100%	0%
0.1%	7168	75%	7.2	98.0%	100%
0.5%	7168	75%	36.0	56.3%	100%
1%	7168	75%	71.7	14.3%	100%

2%	7168	75%	142.8	0%	100%
----	------	-----	-------	----	------

El overhead de 75% es constante (esperado: cada bloque de 4 bits de datos ocupa 7 bits codificados, 3/4 de redundancia) e independiente de la tasa de error, ya que depende únicamente del tamaño del mensaje. La recuperación exacta (frame recibido igual al original, sin necesidad de corrección) cae rápidamente al subir la tasa de error, mientras que la tasa de corrección exitosa se mantiene en 100% en casi todos los casos de esta tabla, porque Hamming(7,4) corrige hasta un bit erróneo por bloque de 7 — con tasas de error bajas y bloques pequeños, es infrecuente que un mismo bloque reciba dos o más bits volteados (caso no corregible).

8. Discusión

El enunciado y la propuesta conjunta original consideraban la IP como identificador único de nodo. En pruebas locales sobre 127.0.0.1 esto es insuficiente, ya que todos los nodos comparten la misma IP y solo se distinguen por el puerto. Se adoptó ip:puerto como identificador canónico en todo el protocolo (LSA, tabla de enrutamiento, mensajes DATA), decisión que se mantiene también en Tailscale sin costo adicional, ya que ahí la IP por sí sola ya era única.

Se aplicó Hamming(7,4) de forma independiente a cada bloque, esto multiplica la capacidad de corrección real: un error es corregible mientras no haya dos o más bits volteados dentro del mismo bloque de 7, en vez de en todo el mensaje. El costo del overhead teórico (75%) en el esquema.

Retomar el Cajero/Banco del Lab 2 requirió un rediseño de su manejo de sesión: en el Lab 2 la sesión autenticada vivía implícita en una conexión TCP persistente; en este plano de datos cada DATA es una entrega independiente sin estado de conexión. Se optó por que el banco mantenga la sesión autenticada indexada explícitamente por la dirección del cajero remitente, lo que además permite atender múltiples cajeros concurrentes de forma correcta y aislada.

Se optó por consolidar toda la implementación en Go, lo que simplificó el mantenimiento del código y permitió invertir el tiempo ahorrado en robustecer el plano de control y recuperar la aplicación Cajero/Banco en vez de mantener dos implementaciones paralelas del mismo protocolo.

9. Conclusiones

- Se implementó un protocolo Link State completo y funcional (descubrimiento de vecinos, LSA, flooding, Dijkstra y tabla de enrutamiento), verificado tanto en una topología local determinista como en una red real (Tailscale) interoperando con una implementación independiente de otra pareja.
- El plano de datos, protegido con Hamming(7,4) por bloques, entrega correctamente los mensajes a través de la ruta más corta calculada, incluso ante tasas de error por bit de hasta 1-2%, con overhead constante de 75% independientemente de la tasa de error.
- Separar el protocolo por bloques (en vez de un único código de corrección sobre todo el frame) fue la decisión de diseño con mayor impacto en la tolerancia real a errores del sistema.

- La aplicación de demostración Cajero/Banco confirmó que el sistema completo —control y datos— funciona de extremo a extremo bajo una carga de trabajo con sentido práctico, no solo con mensajes de prueba sintéticos.

Referencias

- Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson. Capítulo 5: The Network Layer (enrutamiento por estado de enlace, algoritmo de Dijkstra).
- Moy, J. (1998). OSPF Version 2. RFC 2328, Internet Engineering Task Force.
<https://www.rfc-editor.org/rfc/rfc2328>
- Hamming, R. W. (1950). Error detecting and error correcting codes. The Bell System Technical Journal, 29(2), 147-160.

[\[Link al Repo\]](#)